



ITA602: Advanced Information Technology
Work Integrated Learning

Assessment 1

Project Review Report of Project Proposal and Planning

Orbis Scheduler

A Multi-Tenant Booking and Scheduling Platform for Service Based Businesses

Submitted By

Hussain Jameel A00180177
Smitkumar Teli A00184801

Table of Contents

Table of Contents	2
1. Proposed Project Title	3
2. Problem Statement	3
2.1 Gaps	3
2.2 Expected Outcomes	3
3. Background	4
3.1 Digital Transformation	4
3.2 Importance	4
3.3 Relevance to Contemporary ICT Practices	4
4. Scope	5
4.1 In Scope	5
4.2 Out of Scope	5
4.3 Stakeholders	6
5. Framework	6
5.1 Why Agile Scrum	6
5.2 Sprint Activities and Implementation	6
5.3 Benefits	6
6. Method	7
6.1 Requirements Analysis	7
6.2 System Design	7
6.3 Development	7
6.4 Testing	7
6.5 Documentation and Version Control	7
7. Tentative Project Timeline	8
8. Limitations	8
9. Delimitations	9
10. Communication Management Plan	9
10.1 Team	9
10.2 Communication Channels	10
10.3 Meeting Schedule	10
10.4 Reporting	10
11. References	10

1. Proposed Project Title

Orbis Scheduler: A Multi-Tenant Booking and Scheduling Platform for Service Based Businesses

We are developing Orbis Scheduler, a booking and scheduling platform that lets several independent service businesses run their appointments through one shared system while keeping each business's data completely separate from the others. The project gives business owners a dashboard to manage their availability and bookings, gives their customers a simple way to request a time slot online and gives us, as the development team, a solid technical project to demonstrate what we have learned across the course.

2. Problem Statement

2.1 Gaps

Most small service businesses like tutors, consultants, cleaners and similar trades still run their bookings the old fashioned way. They take calls, reply to messages and keep a paper diary or a shared spreadsheet up to date by hand. This works up to a point but it falls apart quickly once a business gets a few customers a day. Double bookings happen, requests get missed and the owner ends up spending time on admin instead of doing the actual work that earns them money.

There are booking platforms on the market already, but most of them are either built for large enterprises with pricing and setup that does not suit a one or two person business or they are cheap tools that do not take data separation seriously. When a platform hosts many businesses on shared infrastructure without strong boundaries between them, a flaw in one part of the system can expose customer data that belongs to a completely different business. This is the gap we are aiming at: a booking platform that is simple enough for a small business to set up themselves but that still treats tenant isolation and data protection as a core part of the design rather than an afterthought.

2.2 Expected Outcomes

The primary objective of the Orbis Scheduler project is to research, design and deploy a minimum viable product (MVP) multi-tenant scheduling application. A business owner should be able to register, set their weekly availability, build a simple intake form for their customers and manage incoming bookings from one dashboard. A customer should be able to find a business's public booking page, fill in the form and request a time without needing to create an account.

On the practical side this gives small businesses a real alternative to managing bookings by hand. Business owners will realize immediate operational efficiency by transitioning from fragmented booking processes to a centralized dashboard. The system's embeddable iframe widget will allow businesses to integrate the booking pipeline directly into their existing web presence, reducing customer friction and lowering the administrative overhead associated with managing scheduling conflicts.

On the more theoretical side, the project lets us demonstrate how a Node.js backend, a PostgreSQL database and JWT based authentication can be combined to support multiple tenants securely on one set of infrastructure which is exactly the kind of architecture used across the SaaS industry today. Multi-tenant booking systems are a well established pattern across that industry, the same basic idea sits behind tools used for everything from salon bookings to healthcare scheduling so building one ourselves gives us hands on experience with patterns we are likely to run into in real jobs. It also gives us a chance to build and test our own defences against attacks like Insecure Direct Object Reference, where someone tries to reach another tenant's data by changing an identifier in a request rather than just reading about that risk in a textbook.

3. Background

3.1 Digital Transformation

Booking and scheduling software has become one of the more visible parts of the wider SaaS market over the past several years. The businesses offering it have shown that customers are happy to book everything from a haircut to a tutoring session through a website rather than a phone call. That demand has pulled a lot of small operators online who previously had no digital booking system at all, which is exactly the audience Orbis Scheduler is aimed at.

Customers now expect to book things online without having to call during business hours. The businesses that offer this tend to win out over the ones that do not. That shift has been good for customer convenience, but it also means businesses are now responsible for storing personal information online, things like names, phone numbers, emails and appointment history that they used to keep privately on paper or on their own phone.

3.2 Importance

Small and medium-sized businesses require solutions that simplify appointment management while actively protecting their business intelligence and client data. This matters more for small businesses than people sometimes assume, because most of them do not have an IT person on staff to think about security. If a shared booking platform is not built carefully, a single weak point can affect every business using it, not just one. That is why we are treating multi-tenant security as a core requirement of Orbis Scheduler rather than something we will deal with later once the main features are working.

3.3 Relevance to Contemporary ICT Practices

The project aligns comfortably within current ICT practice. We are using a multi-tenant SaaS architecture, where many businesses share one application and one database while staying logically separated from each other. Authentication runs through JSON Web Tokens, which is the standard way modern web APIs handle stateless authentication without keeping session state on the server (Jones et al., 2015). We are also applying role based access control so that what a customer, a business owner and a platform administrator can each do is clearly separated. We are following the OWASP Top 10: 2021 guidance around broken access control as well, since that risk sits closest to the kind of tenant isolation failure we are trying to avoid. We are paying similar attention to injection and security misconfiguration risks too, given how much of the system depends on form data submitted by customers (OWASP Foundation, 2021). On the data protection side, we are using the Australian Privacy Principles and the idea of Privacy by Design to guide how much customer information we actually collect and how long we keep it (Cavoukian, 2009).

4. Scope

The features we are building cover the full booking journey for both sides of the platform.

4.1 In Scope

- Secure registration and login for business owners using JWT for authentication
- A multi-tenant middleware layer that scopes every database query to the business identified in the token using UUIDs rather than sequential IDs so businesses cannot be enumerated
- Weekly availability rules per business with available time slots worked out on demand rather than stored ahead of time
- One flat dynamic booking form per business supporting text fields, text areas, dropdowns, checkboxes and radio buttons
- A public booking page for each business plus an embeddable widget they can place on their own website
- Email based confirmation messages to customers
- A request and approval booking flow where business owners accept or reject incoming requests from their dashboard
- A platform administrator dashboard covering admin login, a platform overview, a list of all registered businesses and the ability to suspend or activate a business

The embeddable widget matters because it lets a business keep customers on their own website instead of sending them somewhere else to book, which is closer to how most small businesses already present themselves online.

4.2 Out of Scope

- Native mobile apps for iOS or Android
- Payment processing through Stripe, PayPal or any other gateway
- Automated SMS notifications
- AI based recommendation features
- Multiple service types inside a single business account, since Orbis Scheduler is about supporting many businesses, not many services per business
- Time zone conversion, since the system assumes a business and its customers are in the same local time zone

4.3 Stakeholders

The people who matter most to this project are the business owners using it day to day and their customers booking through it. We are the platform administrators in this version of the project, responsible for keeping the system running and managing business accounts. Our learning facilitator is also a stakeholder, in the sense that the project needs to meet the standards expected for this subject.

5. Framework

5.1 Why Agile Scrum

To systematically guide the research, development and delivery of the Orbis Scheduler, the team will utilize the Agile Scrum project management methodology. Agile facilitates iterative and incremental development. With a team of two members working across the full stack, we do not have the spare capacity to spend weeks on upfront documentation before any code gets written. We wanted a structure that lets us notice problems early rather than near the deadline.

5.2 Sprint Activities and Implementation

The 10-week development window will be divided into one-week sprints. Each sprint will follow a defined lifecycle:

- **Sprint Planning:** The team will select high priority user cases from the product backlog (e.g. configuring availability, generating JWTs).
- **Development:** Parallel execution of frontend and backend tasks.
- **Testing:** Continuous integration testing and vulnerability scanning (checking for authorization bypasses) at the end of each sprint.
- **Review and Retrospective:** Weekly analysis of code quality, merge conflicts and team velocity.

5.3 Benefits

Implementing Agile Scrum is highly beneficial for a development team of few members. It provides a flexible development process that can pivot when technical blockers (such as database routing issues) arise. It ensures continuous feedback and allows for incremental improvements, significantly reducing the risk of a catastrophic architectural failure late in the academic term.

6. Method

The project will be executed following a structured Software Development Life Cycle (SDLC) tailored to our Agile framework:

6.1 Requirements Analysis

The initial phase involves a thorough review of the project proposal to define explicit user requirements. The team will analyze the business needs of theoretical service providers and translate these into 13 discrete technical use cases, mapping out the exact data flow between the customer interface, the backend API, and the PostgreSQL database.

6.2 System Design

This phase focuses on the architectural blueprint of the application. The team will construct the database Entity-Relationship Diagrams (ERDs), establishing how tables will relationalize while maintaining tenant isolation. Use case diagrams will be formalized, and wireframes for the React.js dashboard and customer booking widgets will be drafted to establish a clear user experience (UX) baseline.

6.3 Development

Development will be executed across three functional domains:

- **Business Owner Features:** Engineering the core SaaS logic. This includes developing secure registration endpoints, building the dynamic form configuration tool using JSON schemas, and implementing the availability management engine.
- **Customer Features:** Developing the unauthenticated, public-facing booking flow. This involves creating the algorithms that calculate available time slots based on the business's configuration and processing the final booking submission safely.
- **Administrator Features:** Building an isolated, high-privilege dashboard that allows platform administrators to query the entire database across tenants for monitoring purposes and to suspend malicious or inactive business accounts.

6.4 Testing

Quality assurance will occur iteratively. Functional testing will ensure all 13 use cases operate as designed. Security testing will be paramount; the team will manually test API endpoints to ensure IDOR vulnerabilities are mitigated and that input validation successfully blocks SQL injection attempts. User acceptance testing (UAT) will be simulated to ensure the booking flow is intuitive.

6.5 Documentation and Version Control

The final phase will synthesize the development process into academic and technical artifacts. This includes commenting the codebase, finalizing the API endpoint specifications, and preparing the final project report for academic submission.

7. Tentative Project Timeline

We have ordered the weeks so the security foundations, authentication and the multi-tenant middleware, are in place before we build anything that depends on them, with testing and polish kept for the final two weeks rather than squeezed in alongside new features.

Week	Activity
Week 1	Use case finalisation and wireframing for the booking form and dashboards
Week 2	Literature review, Repository setup, Git workflow configuration and the initial database schema design
Week 3	System Architecture planning, setting up Node.js and Express environment setup and PostgreSQL provisioning
Week 4	Development of JWT Authentication protocols and Business Owner registration logic.
Week 5	Implementation of the Multi-tenant middleware layer and Service CRUD (Create, Read, Update, Delete) operations.
Week 6	Dynamic form builder and public booking page (React and Next.js)
Week 7	Availability Management logic and algorithmic slot calculation implementation.
Week 8	Admin dashboard and business owner dashboard refinement.
Week 9	Comprehensive Functional and Security Testing, bug resolution, and code refactoring.
Week 10	Final Technical Documentation, user manual creation and project submission preparation.

8. Limitations

The execution of this project is subject to several unavoidable limitations:

Academic Project Duration: The strict 10-week timeline limits the capacity to implement complex automated testing pipelines (CI/CD) or conduct extensive, real-world beta testing with actual businesses.

Team Capacity: The full-stack development, database architecture and security testing must be entirely managed by a two person student team.

Budget and Infrastructure: The project relies on free-tier cloud hosting solutions (e.g, AWS Free Tier, Heroku, or Vercel). These environments impose strict limitations on database storage, compute power and bandwidth which will restrict the team's ability to perform high-volume load testing.

9. Delimitations

To ensure the successful delivery of the MVP, strict project boundaries have been established:

- A responsive web application only, with no native mobile app planned
- Built to reasonable security standards for an academic project, but not put through a formal security audit or a real penetration test
- Tested using simulated business and customer data rather than real client data
- One business model per tenant rather than letting a single business offer multiple distinct service types, which keeps the booking form and the availability logic much simpler to build correctly
- Payments, AI recommendations and SMS notifications left out deliberately, so we can spend our limited time getting the booking flow and the tenant isolation right rather than spreading ourselves thin

Within these boundaries, we think the plan in this report gives us a realistic path to a working product by the end of the trimester.

10. Communication Management Plan

10.1 Team

Team Member	Role	Responsibilities
Hussain Jameel	Project Lead	- Requirements gathering - System architecture - Use case development - Project coordination
Smitkumar Teli	Security Analyst , Documentation and Research Coordinator	- Project documentation - Literature review - Security and ethics analysis - Testing support

10.2 Communication Channels

- WhatsApp Group
- Microsoft Teams
- Email
- Github

10.3 Meeting Schedule

Sprint	Frequency	Duration	Purpose
Weekly Team Meeting	Once per week	1 Hour	- Progress review - Task allocation - Issue resolution

10.4 Reporting

Decisions will be made via consensus. Weekly progress updates will be documented and stored in a shared cloud repository (Google Drive/Github) to maintain an audit trail of the project's evolution. Technical version control will be strictly managed via GitHub providing a transparent reporting mechanism for code commits and feature completions. Any major disputes or significant technical hurdles that cannot be resolved internally will be reported to the Learning Facilitator for guidance.

11. References

Cavoukian, A. (2009). Privacy by design: The 7 foundational principles. *Information and privacy commissioner of Ontario, Canada*, 5(2009), 12.

Schwaber, K., & Sutherland, J. (2020). The Scrum guide: The definitive guide to Scrum: The rules of the game. *Scrum.org*, 133-152.

Jones, M., Bradley, J., & Sakimura, N. (2015). Rfc 7519: Json web token (jwt).

Policy, A. (2014). Australian Privacy Principles.

OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/2021/>